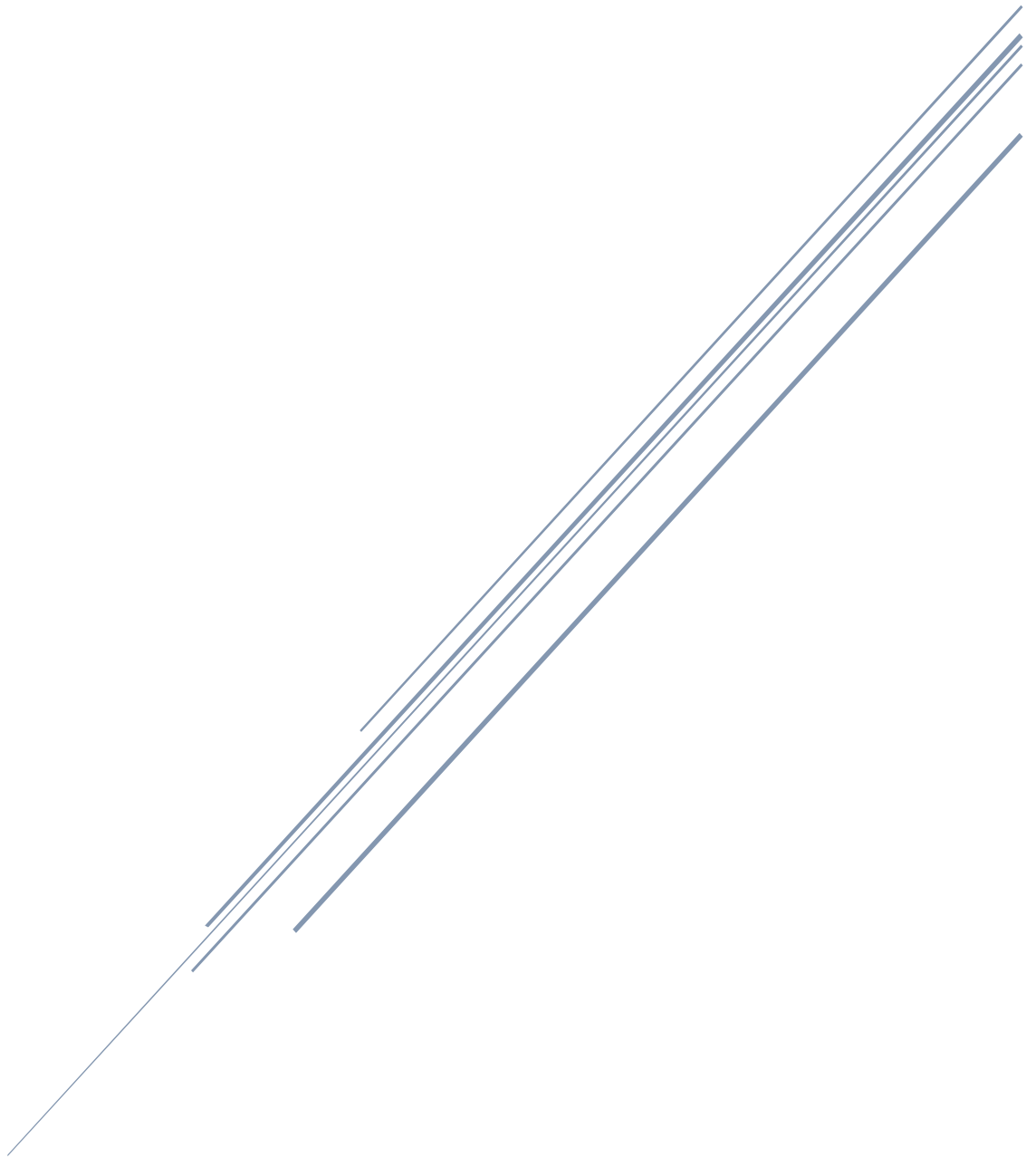


CLOUD COMPUTING

MUHAMMAD WAQAS SALEEM



SEMESTER PROJECT

Amina Noor(007), Hamail Fatima(023), Hajra Sarwar(022)

Table of Contents

1. Project Introduction.....	2
2. Project Objectives.....	2
3. Tools & Technologies Used.....	2
4. Architecture Overview	2
4.1. High-Level Architecture Description	2
4.2. Architecture Diagram	3
5. Git Repository Setup	3
5.1. Repository Structure	3
5.2. Git Branching Strategy	5
5.3. .gitignore Configuration.....	6
6. Terraform Infrastructure Provisioning.....	6
6.1. Network Module – VPC & Subnets	6
6.2. Load Balancer EC2 Module	7
6.3. Web EC2 Module	8
7. Environment Configuration	9
7.1. Root Terraform Configuration	9
7.2. Environment-Specific Variables	9
8. Ansible Configuration & Automation	12
8.1. Ansible Inventory & Configuration	12
8.2. Web Stack Configuration (Nginx + PHP-FPM)	12
8.3. Load Balancer Configuration.....	13
8.4. Website Deployment & Onboarding Automation	14
9. Monitoring & Customer Onboarding.....	17
9.1. Monitoring Scripts	17
9.2. Customer Onboarding Documentation	17
10. Conclusion	18

PROJECT 6 – HIGH-PERFORMANCE WEB HOSTING PLATFORM

1. Project Introduction

Modern web applications require **high availability, scalability, performance, and automation**.

This project focuses on designing and implementing a **production-ready, high-performance web hosting platform** on AWS using **Infrastructure as Code (Terraform)** and **Configuration Management (Ansible)**.

The platform hosts **multiple customer websites (multi-tenancy)** using **Nginx virtual hosts**, provides **load balancing, SSL termination, performance tuning, and automated onboarding**, while staying strictly within the tools covered in the course.

2. Project Objectives

The main objectives of this project are:

- Design a **multi-AZ web hosting architecture** using AWS EC2 and VPC.
 - Use **Terraform modules** to provision infrastructure in a reusable and environment-aware manner.
 - Use **Ansible roles and playbooks** to automate:
 - Web stack installation (Nginx + PHP-FPM)
 - Website deployment
 - SSL configuration
 - Monitoring
 - Implement **Nginx load balancing** instead of AWS ALB.
 - Host **multiple customer websites** using Nginx server blocks.
 - Provide **automated onboarding** for adding new customer websites.
 - Implement **basic monitoring scripts** for uptime and response time.
-

3. Tools & Technologies Used

- **AWS:** IAM, VPC, EC2, Security Groups, NAT Gateway (borderline)
 - **Terraform:** Infrastructure provisioning & modules
 - **Ansible:** Configuration management & automation
 - **Nginx:** Load balancing, reverse proxy, SSL, caching
 - **PHP-FPM:** PHP runtime
 - **Linux (Ubuntu):** Server OS
 - **Git & GitHub:** Version control & collaboration
 - **Bash:** Monitoring scripts
-

4. Architecture Overview

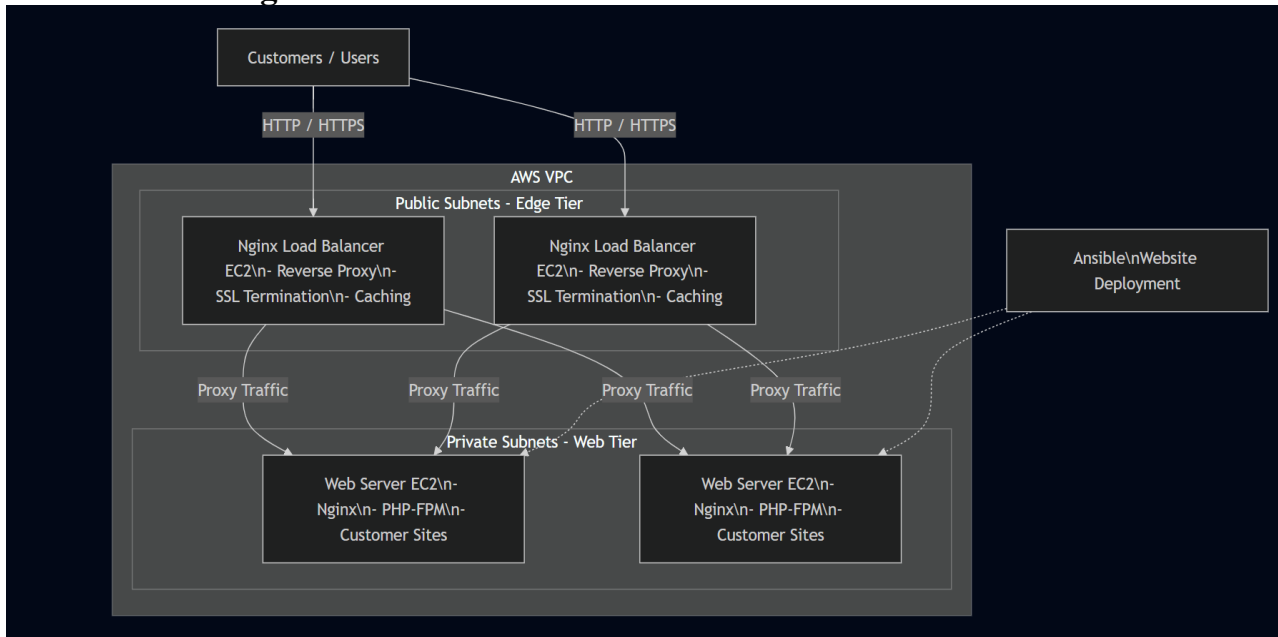
4.1 High-Level Architecture Description

The platform follows a **two-tier architecture** deployed across **two Availability Zones**:

- **Public Subnets (Edge Tier)**
Nginx Load Balancer EC2 instances handle:
 - Incoming HTTP/HTTPS traffic
 - SSL termination
 - Caching
 - Load balancing
- **Private Subnets (Web Tier)**
Web server EC2 instances run:
 - Nginx
 - PHP-FPM
 - Multiple customer websites using server blocks

Website content is deployed using **Ansible**, ensuring **consistent content across all web servers without NFS**.

4.2 Architecture Diagram

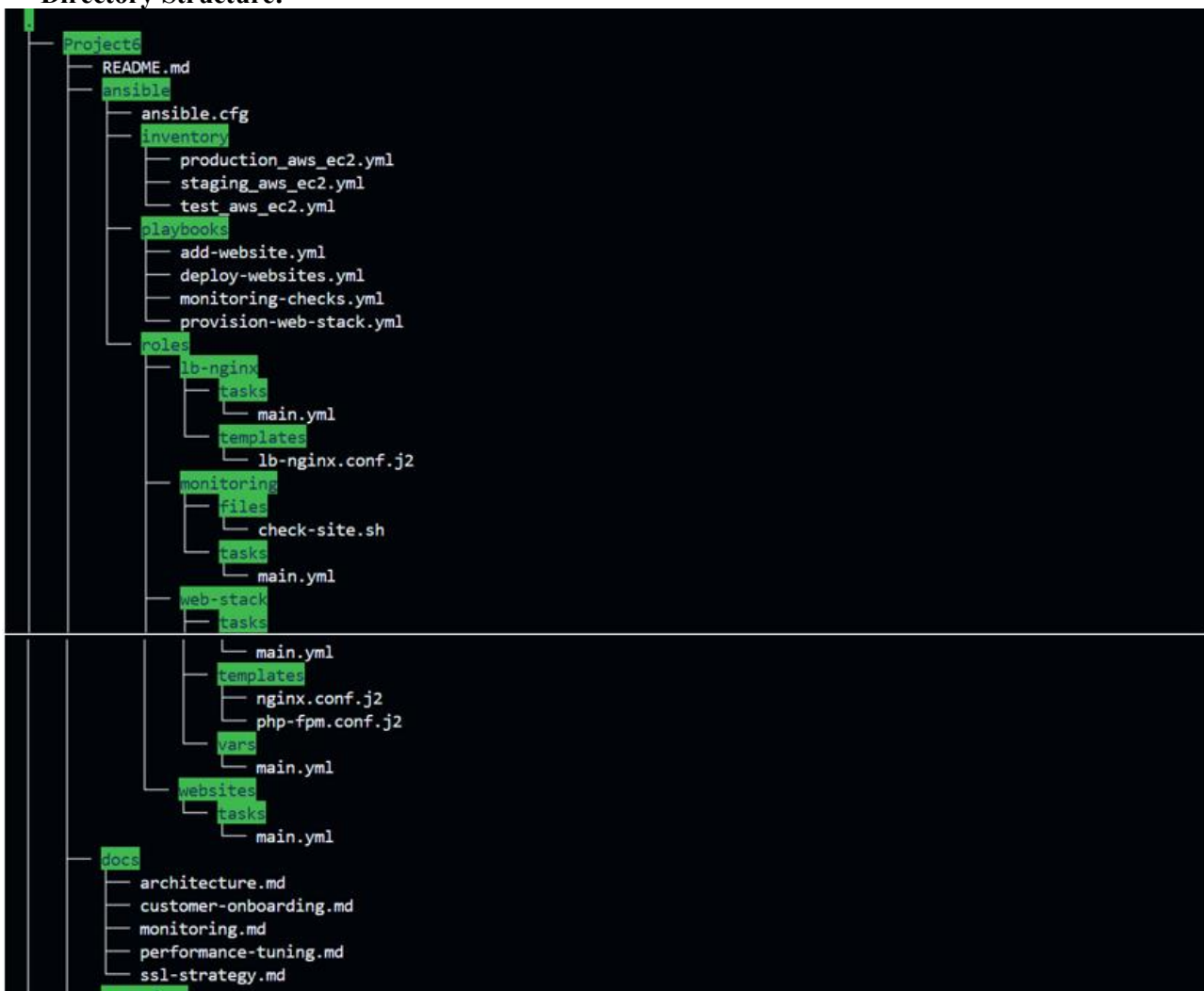


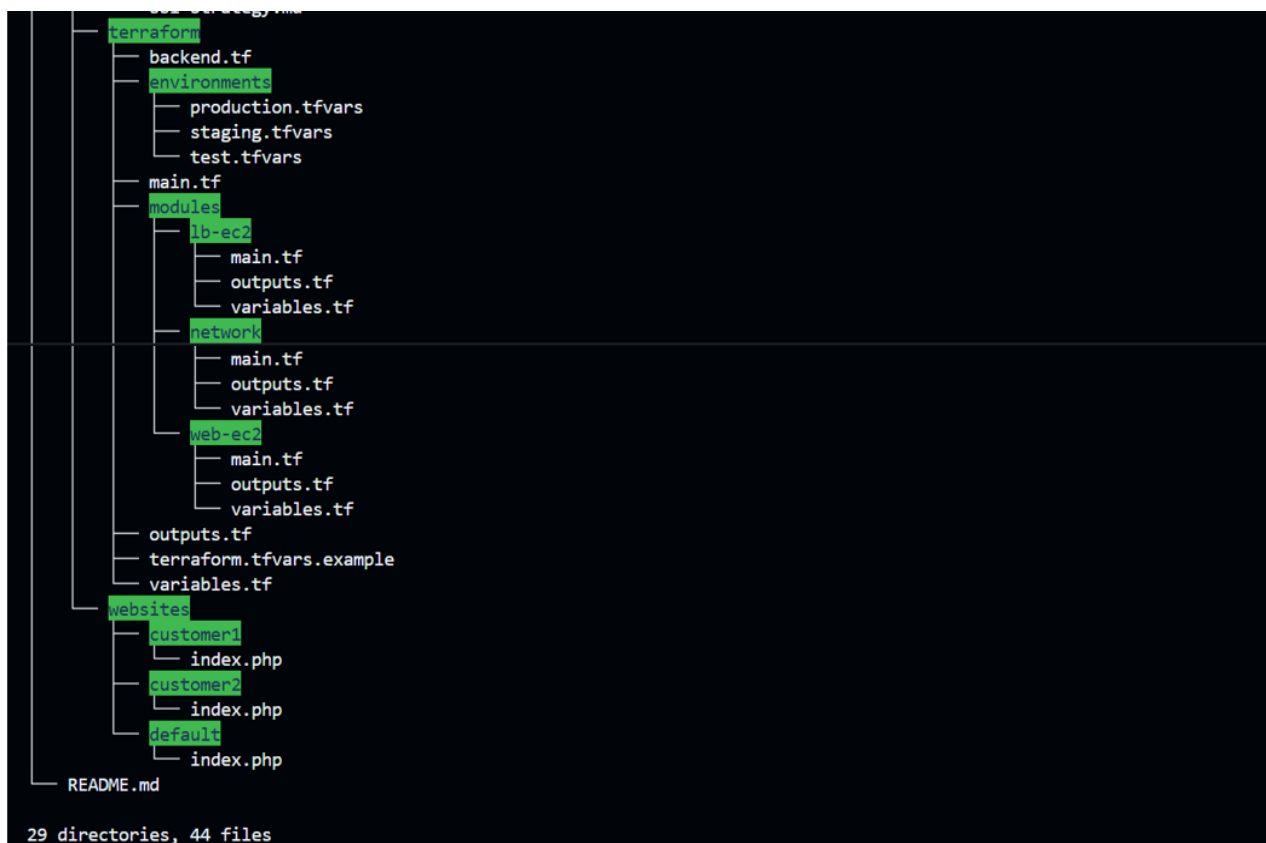
5. Git Repository Setup

5.1. Repository Structure

A structured Git repository is created to separate **Terraform**, **Ansible**, **websites**, and **documentation**.

Directory Structure:





Steps:

1. Create the directory structure

```

@23-22411-007-oss →/workspaces/CC-Project/Project6 (main) $ mkdir -p terraform/environments
mkdir -p terraform/modules/network
mkdir -p terraform/modules/lb-ec2
mkdir -p terraform/modules/web-ec2
@23-22411-007-oss →/workspaces/CC-Project/Project6 (main) $
  
```

```

@23-22411-007-oss →/workspaces/CC-Project/Project6 (main) $ mkdir -p terraform/environments
mkdir -p terraform/modules/network
mkdir -p terraform/modules/lb-ec2
mkdir -p terraform/modules/web-ec2
  
```

```

@23-22411-007-oss →/workspaces/CC-Project/Project6 (main) $ mkdir -p ansible/inventory
mkdir -p ansible/playbooks
mkdir -p ansible/roles/web-stack/tasks
mkdir -p ansible/roles/web-stack/templates
mkdir -p ansible/roles/web-stack/vars
mkdir -p ansible/roles/lb-nginx/tasks
mkdir -p ansible/roles/lb-nginx/templates
mkdir -p ansible/roles/websites/tasks
mkdir -p ansible/roles/monitoring/tasks
mkdir -p ansible/roles/monitoring/files
@23-22411-007-oss →/workspaces/CC-Project/Project6 (main) $
  
```

```

@23-22411-007-oss →/workspaces/CC-Project/Project6 (main) $ mkdir -p websites/customer1
mkdir -p websites/customer2
mkdir -p websites/default
  
```

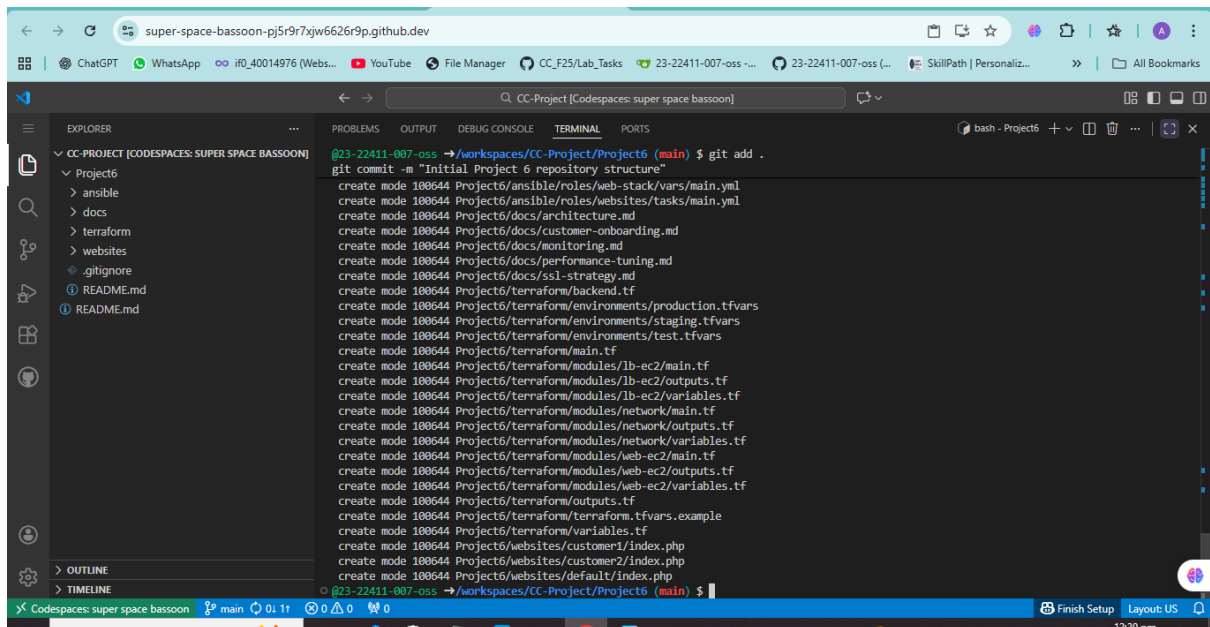
```

touch websites/customer1/index.php
touch websites/customer2/index.php
touch websites/default/index.php
  
```

```

@23-22411-007-oss →/workspaces/CC-Project/Project6 (main) $ mkdir -p docs
touch docs/architecture.md
touch docs/performance-tuning.md
touch docs/ssl-strategy.md
touch docs/monitoring.md
touch docs/customer-onboarding.md
  
```

2. Initialize Git repository
3. Add initial files and commit



5.2 Git Branching Strategy

A multi-environment branching strategy is followed:

- main → Production
- staging → Pre-production
- test → Internal testing
- feature/* → New features

Steps:

1. Create branches

```

@hajrasarwar11 → /workspaces/CC-Project (main) $ git branch --list
* main
@hajrasarwar11 → /workspaces/CC-Project (main) $ git branch test
@hajrasarwar11 → /workspaces/CC-Project (main) $ git branch staging
@hajrasarwar11 → /workspaces/CC-Project (main) $ git branch --list
* main
  staging
  test
  
```

2. Document strategy in README.md

```

Project6 > README.md > abc # Git Branching Strategy
1  # Git Branching Strategy
2
3  This project uses a branching model inspired by Project1:
4
5  - **main**: Production platform (deploys to production after approvals)
6  - **staging**: Pre-production (deploys to staging after successful testing)
7  - **test**: Internal testing (deploys to test environment)
8  - **feature/**: New features or improvements (merged into test via PR and
  review)
9
10 **Rules:**
11 - `feature/*` branches are merged into `test` via pull request and review.
12 - `test` branch is deployed to the test environment.
13 - `staging` branch is deployed to the staging environment after successful
  testing.
14 - `main` branch is deployed to production after approvals.
15
16 ...
17 main (Production platform)
18 |
19 | └─ staging (Pre-prod)
20 |   └─ test (Internal testing)
     └─ feature/* (New features / improvements)
  
```

5.3 .gitignore Configuration

A .gitignore file is added to prevent committing:

- Terraform state files
- AWS credentials
- SSH keys
- Logs and temp files

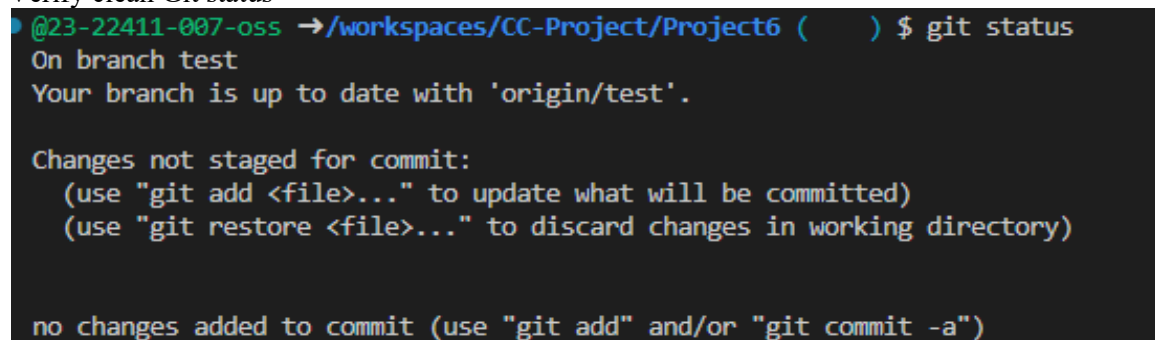
Steps:

1. Create .gitignore



```
.gitignore M X
Project6 > .gitignore
1 # Terraform
2 **/.terraform/*
3 *.tfstate
4 *.tfstate.*
5 *.tfvars
6 !*.tfvars.example
7 crash.log
8 crash.*.log
9 .terraformrc
10 terraform.rc
11
12 # Ansible
13 *.retry
14 *.secret
15
16 # AWS credentials
17 .aws/
18 *.pem
19 *.key
20
21 # IDE
```

2. Verify clean Git status



```
@23-22411-007-oss →/workspaces/CC-Project/Project6 ( ) $ git status
On branch test
Your branch is up to date with 'origin/test'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)

no changes added to commit (use "git add" and/or "git commit -a")
```

6. Terraform Infrastructure Provisioning

6.1 Network Module – VPC & Subnets

A reusable Terraform **network module** provisions:

- VPC (10.0.0.0/16)
- Public subnets (Load Balancers)
- Private subnets (Web Servers)
- Internet Gateway
- Route tables
- NAT Gateway (if required)

Steps:

1. Define VPC and subnets in `modules/network/main.tf`

```
.gitignore M  outputs.tf M  variables.tf M  main.tf M X
Project6 > terraform > modules > network > main.tf
1  # VPC
2  resource "aws_vpc" "main" {
3      cidr_block = var.vpc_cidr
4      enable_dns_support = true
5      enable_dns_hostnames = true
6      tags = {
7          Name = "project6-vpc"
8      }
9  }
10
11 # Public Subnets
12 resource "aws_subnet" "public" {
13     count = length(var.public_subnet_cidrs)
14     vpc_id = aws_vpc.main.id
15     cidr_block = var.public_subnet_cidrs[count.index]
16     availability_zone = var.azs[count.index]
17     map_public_ip_on_launch = true
18     tags = {
19         Name = "project6-public-${count.index + 1}"
20         Tier = "public"
21     }
22 }
23
24 # Private Subnets
25 resource "aws_subnet" "private" {
```

2. Export VPC and subnet IDs via outputs

```
.gitignore M  outputs.tf M X  variables.tf M  main.tf M
Project6 > terraform > modules > network > outputs.tf
1  output "vpc_id" {
2      description = "VPC ID"
3      value = aws_vpc.main.id
4  }
5
6  output "public_subnet_ids" {
7      description = "List of public subnet IDs"
8      value = [for s in aws_subnet.public : s.id]
9  }
10
11 output "private_subnet_ids" {
12     description = "List of private subnet IDs"
13     value = [for s in aws_subnet.private : s.id]
14 }
```

6.2 Load Balancer EC2 Module

Two EC2 instances are provisioned as **Nginx Load Balancers** across two AZs.

Key Features:

- Public subnets
- Ports 80/443 open to internet
- SSH restricted to admin IP
- Tagged with Role=lb

Steps:

1. Create LB module configuration

```
.gitignore M  main.tf M X  outputs.tf M  variables.tf M
Project6 > terraform > modules > lb-ec2 > main.tf
1  # Security Group for LBs
2  resource "aws_security_group" "lb" {
3      name = "lb-sg"
4      description = "Allow HTTP, HTTPS from anywhere; SSH from admin only"
5      vpc_id = var.vpc_id
6
7      ingress {
8          description = "HTTP"
9          from_port = 80
10         to_port = 80
11         protocol = "tcp"
```


2. Output public and private Ips

```
Project6 > terraform > modules > lb-ec2 > outputs.tf
1  output "lb_public_ips" {
2      description = "Public IPs of LB EC2 instances"
3      value       = [for i in aws_instance.lb : i.public_ip]
4  }
5
6  output "lb_public_dns" {
7      description = "Public DNS names of LB EC2 instances"
8      value       = [for i in aws_instance.lb : i.public_dns]
9  }
10
11 output "lb_private_ips" {
12     description = "Private IPs of LB EC2 instances"
13     value       = [for i in aws_instance.lb : i.private_ip]
14 }
```

6.3 Web EC2 Module

Multiple EC2 instances are provisioned as **web servers** in private subnets.

Key Features:

- Access only from LB
- PHP-FPM enabled
- Tagged with Role=web

Steps:

1. Define web EC2 resources

```
Project6 > terraform > modules > web-ec2 > main.tf
1  # Security Group for Web Servers
2  resource "aws_security_group" "web" {
3      name         = "web-sg"
4      description  = "Allow HTTP from LB SG only; SSH from admin only"
5      vpc_id       = var.vpc_id
6
7      ingress {
8          description = "HTTP from LB SG"
9          from_port   = 80
10         to_port    = 80
11         protocol   = "tcp"
12         security_groups = [var.lb_sg_id]
13     }
14     ingress {
15         description = "SSH from admin"
16         from_port   = 22
17         to_port    = 22
18         protocol   = "tcp"
19     }
```

2. Output private IPs for Nginx upstream

```
Project6 > terraform > modules > web-ec2 > outputs.tf
1  output "web_instance_ids" {
2      description = "IDs of web EC2 instances"
3      value       = [for i in aws_instance.web : i.id]
4  }
5
6  output "web_private_ips" {
7      description = "Private IPs of web EC2 instances"
8      value       = [for i in aws_instance.web : i.private_ip]
9  }
```

7. Environment Configuration

7.1. Root Terraform Configuration

The root Terraform configuration manages the **overall infrastructure orchestration** by configuring the AWS provider and calling all required Terraform modules.

Key Responsibilities:

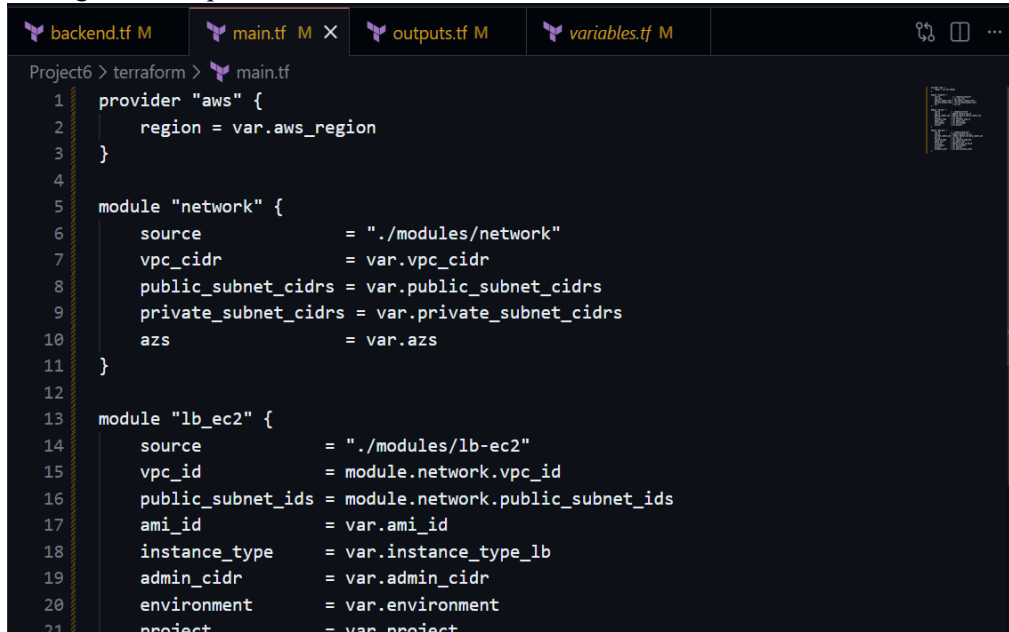
- Configure AWS provider
- Instantiate network, load balancer, and web server modules
- Export infrastructure outputs for Ansible and verification

Exports:

- Load Balancer public IPs
- Web server private IPs
- SSH helper commands (optional)

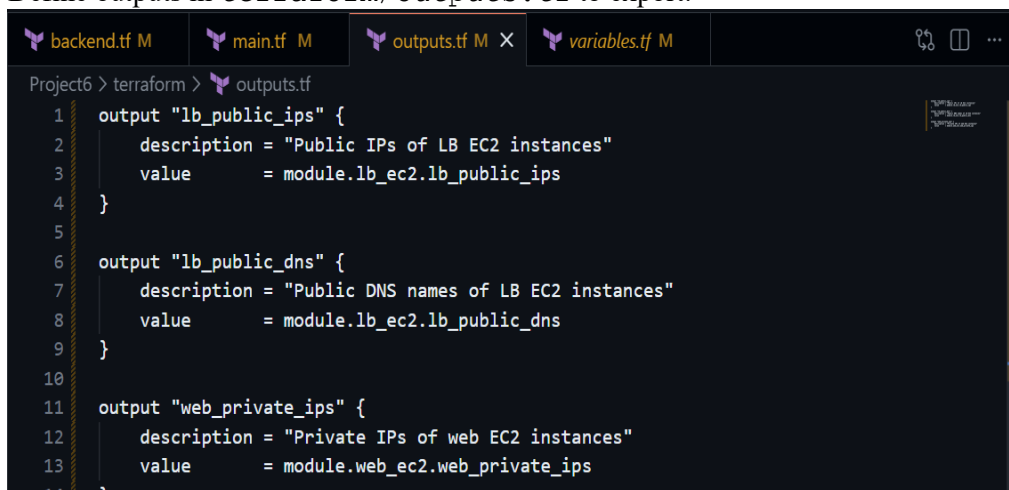
Steps:

1. Configure AWS provider in terraform/main.tf



```
Project6 > terraform > main.tf
1  provider "aws" {
2      region = var.aws_region
3  }
4
5  module "network" {
6      source          = "../modules/network"
7      vpc_cidr        = var.vpc_cidr
8      public_subnet_cidrs = var.public_subnet_cidrs
9      private_subnet_cidrs = var.private_subnet_cidrs
10     azs              = var.azs
11 }
12
13 module "lb_ec2" {
14     source          = "../modules/lb-ec2"
15     vpc_id          = module.network.vpc_id
16     public_subnet_ids = module.network.public_subnet_ids
17     ami_id          = var.ami_id
18     instance_type    = var.instance_type_lb
19     admin_cidr       = var.admin_cidr
20     environment      = var.environment
21     project          = var.project
```

2. Instantiate the **network module** to provision VPC and subnets
3. Instantiate the **lb-ec2 module** for Nginx load balancer instances
4. Instantiate the **web-ec2 module** for private web servers
5. Define outputs in terraform/outputs.tf to export:



```
Project6 > terraform > outputs.tf
1  output "lb_public_ips" {
2      description = "Public IPs of LB EC2 instances"
3      value       = module.lb_ec2.lb_public_ips
4  }
5
6  output "lb_public_dns" {
7      description = "Public DNS names of LB EC2 instances"
8      value       = module.lb_ec2.lb_public_dns
9  }
10
11 output "web_private_ips" {
12     description = "Private IPs of web EC2 instances"
13     value       = module.web_ec2.web_private_ips
14 }
```

- LB public IPs
- Web private IPs

7.2. Environment-Specific Variables

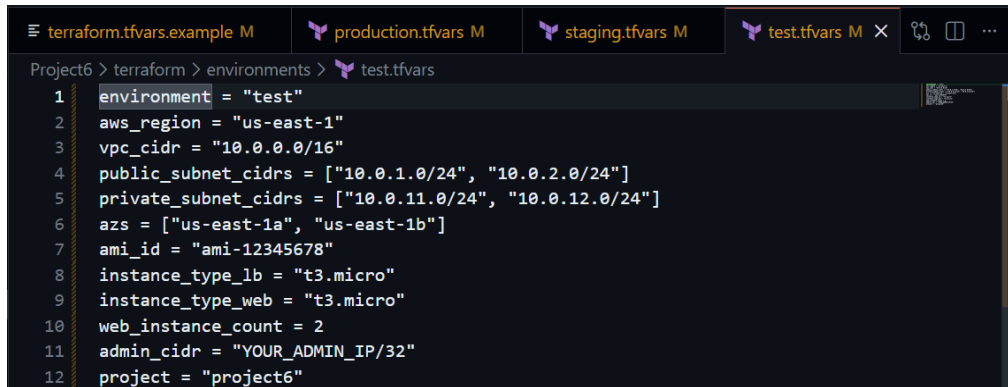
Environment-specific variable files allow the same Terraform codebase to be reused across **test, staging, and production** environments by changing only configuration values.

Each environment defines:

- Environment name
- EC2 instance count
- EC2 instance type
- Allowed administrator IP CIDR

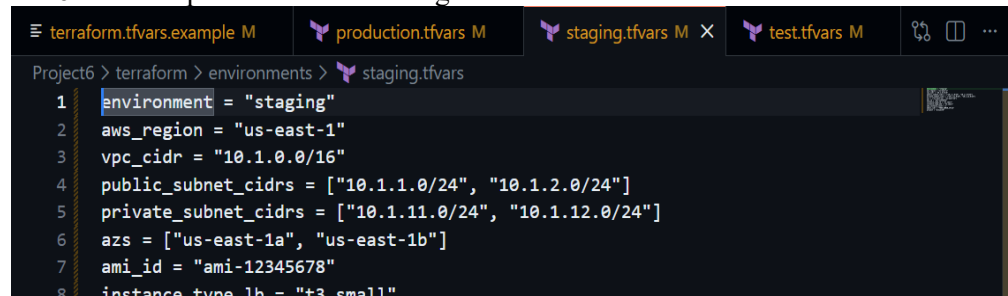
Steps:

1. Create terraform/environments/test.tfvars
 - Define test environment values
 - Use minimal instance size and count



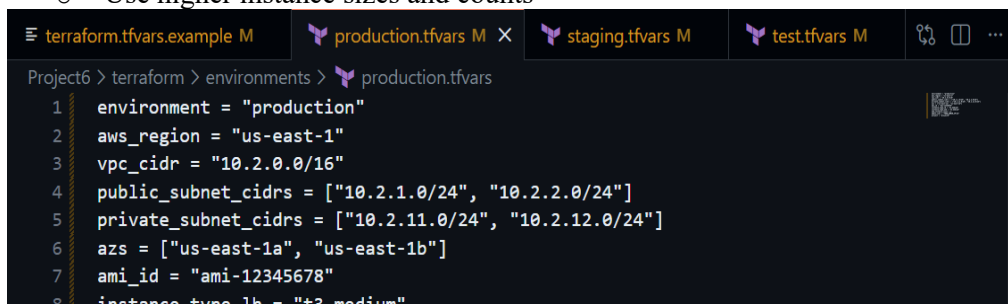
```
1 environment = "test"
2 aws_region = "us-east-1"
3 vpc_cidr = "10.0.0.0/16"
4 public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
5 private_subnet_cidrs = ["10.0.11.0/24", "10.0.12.0/24"]
6 azs = ["us-east-1a", "us-east-1b"]
7 ami_id = "ami-12345678"
8 instance_type_lb = "t3.micro"
9 instance_type_web = "t3.micro"
10 web_instance_count = 2
11 admin_cidr = "YOUR_ADMIN_IP/32"
12 project = "project6"
```

2. Create terraform/environments/staging.tfvars
 - Define staging environment values
 - Match production-like configuration



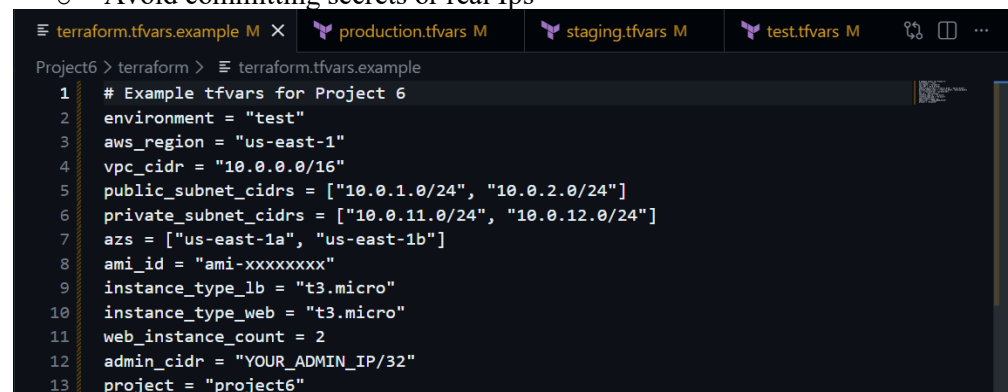
```
1 environment = "staging"
2 aws_region = "us-east-1"
3 vpc_cidr = "10.1.0.0/16"
4 public_subnet_cidrs = ["10.1.1.0/24", "10.1.2.0/24"]
5 private_subnet_cidrs = ["10.1.11.0/24", "10.1.12.0/24"]
6 azs = ["us-east-1a", "us-east-1b"]
7 ami_id = "ami-12345678"
8 instance_type_lb = "t3.small"
```

3. Create terraform/environments/production.tfvars
 - Define production environment values
 - Use higher instance sizes and counts



```
1 environment = "production"
2 aws_region = "us-east-1"
3 vpc_cidr = "10.2.0.0/16"
4 public_subnet_cidrs = ["10.2.1.0/24", "10.2.2.0/24"]
5 private_subnet_cidrs = ["10.2.11.0/24", "10.2.12.0/24"]
6 azs = ["us-east-1a", "us-east-1b"]
7 ami_id = "ami-12345678"
8 instance_type_lb = "t3.medium"
```

4. Create terraform/terraform.tfvars.example
 - Provide sample variable values
 - Avoid committing secrets or real Ips



```
1 # Example tfvars for Project 6
2 environment = "test"
3 aws_region = "us-east-1"
4 vpc_cidr = "10.0.0.0/16"
5 public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
6 private_subnet_cidrs = ["10.0.11.0/24", "10.0.12.0/24"]
7 azs = ["us-east-1a", "us-east-1b"]
8 ami_id = "ami-xxxxxxx"
9 instance_type_lb = "t3.micro"
10 instance_type_web = "t3.micro"
11 web_instance_count = 2
12 admin_cidr = "YOUR_ADMIN_IP/32"
13 project = "project6"
```

AWS Configure:

```
@hajrasarwar11 →/workspaces/CC-Project (main) $ aws configure
AWS Access Key ID [None]: AKIA5WPGPNRI2P4YV6GU
AWS Secret Access Key [None]: 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
Default region name [None]: me-central-1
Default output format [None]: json
```

Terraform init:

```
@hajrasarwar11 →/workspaces/CC-Project/Project6/terraform (main) $ terraform init
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
```

Terraform apply:

```
@hajrasarwar11 →/workspaces/CC-Project/Project6/terraform (main) $ terraform apply
module.network.aws_route.private_nat_gateway: Creation complete after 1s [id=r-rtb-06f80c7936
ed338361080289494]

Apply complete! Resources: 22 added, 0 changed, 0 destroyed.

Outputs:

lb_public_dns = [
  "ec2-51-112-42-23.me-central-1.compute.amazonaws.com",
  "ec2-40-172-99-142.me-central-1.compute.amazonaws.com",
]
lb_public_ips = [
  "51.112.42.23",
  "40.172.99.142",
]
web_private_ips = [
  "10.0.2.87",
  "10.0.3.75",
]
@hajrasarwar11 →/workspaces/CC-Project/Project6/terraform (main) $
```

AWS GUI:

Instances (4) Info Refresh Connect Instance state Actions Launch instances

All states < 1 > Settings

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status
☐	web-2	i-06683d159a421391c	Running	t3.micro	3/3 checks passed	View alarms +
☐	lb-2	i-0eabe3510ac18b658	Running	t3.micro	3/3 checks passed	View alarms +
☐	lb-1	i-0d494f2d6502fe077	Running	t3.micro	3/3 checks passed	View alarms +
☐	web-1	i-0e074aaf1a3e8d5aa	Running	t3.micro	3/3 checks passed	View alarms +

Security Groups (4) Info Refresh Actions Export security groups to CSV Create security group

< 1 > Settings

☐	Name	Security group ID	Security group name	VPC ID	Description
☐	lb-sg	sg-0dcebdfff190875a	lb-sg	vpc-00098878fc4f17b69	Allow HTTP, HTTPS from ...
☐	-	sg-0567ae2c936d1bb1c	default	vpc-05883252ecb09ac3e	default VPC security group
☐	web-sg	sg-0ec5ca515d9f1d6c4	web-sg	vpc-00098878fc4f17b69	Allow HTTP from LB SG o...
☐	-	sg-05f0126aee852a27c	default	vpc-00098878fc4f17b69	default VPC security group

8. Ansible Configuration & Automation

8.1 Ansible Inventory & Configuration

AWS dynamic inventory is used to automatically discover EC2 instances based on tags.

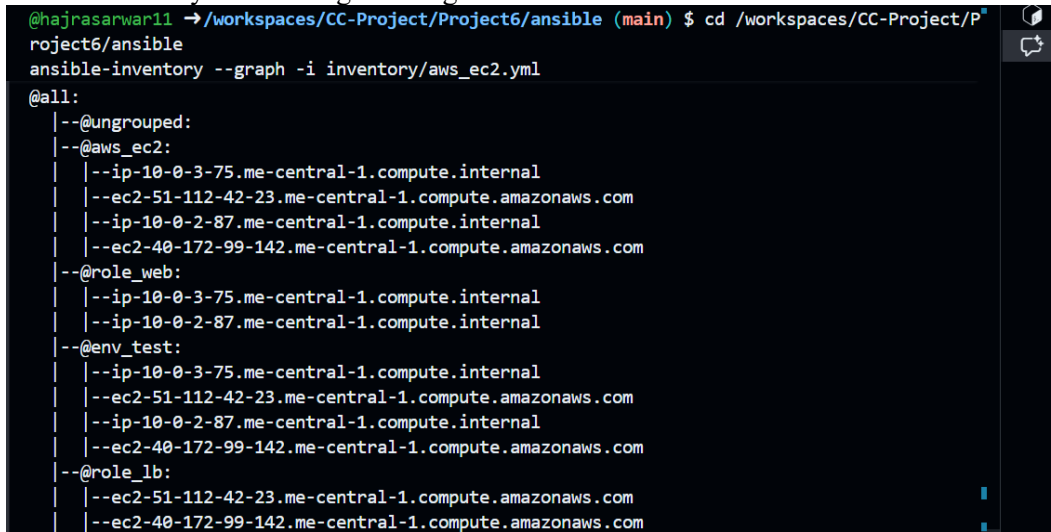
Steps:

1. Configure `ansible.cfg`



```
Project6 > ansible > ansible.cfg
1 [defaults]
2 inventory = ./inventory/aws_ec2.yml
3 host_key_checking = False
4
5 [privilege_escalation]
6 become = True
7 become_method = sudo
8 become_user = root
9 become_ask_pass = False
```

2. Define inventory filters using Role tags



```
@hajrasarwar11 → /workspaces/CC-Project/Project6/ansible (main) $ cd /workspaces/CC-Project/P
project6/ansible
ansible-inventory --graph -i inventory/aws_ec2.yml
@all:
|--@ungrouped:
|--@aws_ec2:
| |--ip-10-0-3-75.me-central-1.compute.internal
| |--ec2-51-112-42-23.me-central-1.compute.amazonaws.com
| |--ip-10-0-2-87.me-central-1.compute.internal
| |--ec2-40-172-99-142.me-central-1.compute.amazonaws.com
|--@role_web:
| |--ip-10-0-3-75.me-central-1.compute.internal
| |--ip-10-0-2-87.me-central-1.compute.internal
|--@env_test:
| |--ip-10-0-3-75.me-central-1.compute.internal
| |--ec2-51-112-42-23.me-central-1.compute.amazonaws.com
| |--ip-10-0-2-87.me-central-1.compute.internal
| |--ec2-40-172-99-142.me-central-1.compute.amazonaws.com
|--@role_lb:
| |--ec2-51-112-42-23.me-central-1.compute.amazonaws.com
| |--ec2-40-172-99-142.me-central-1.compute.amazonaws.com
```

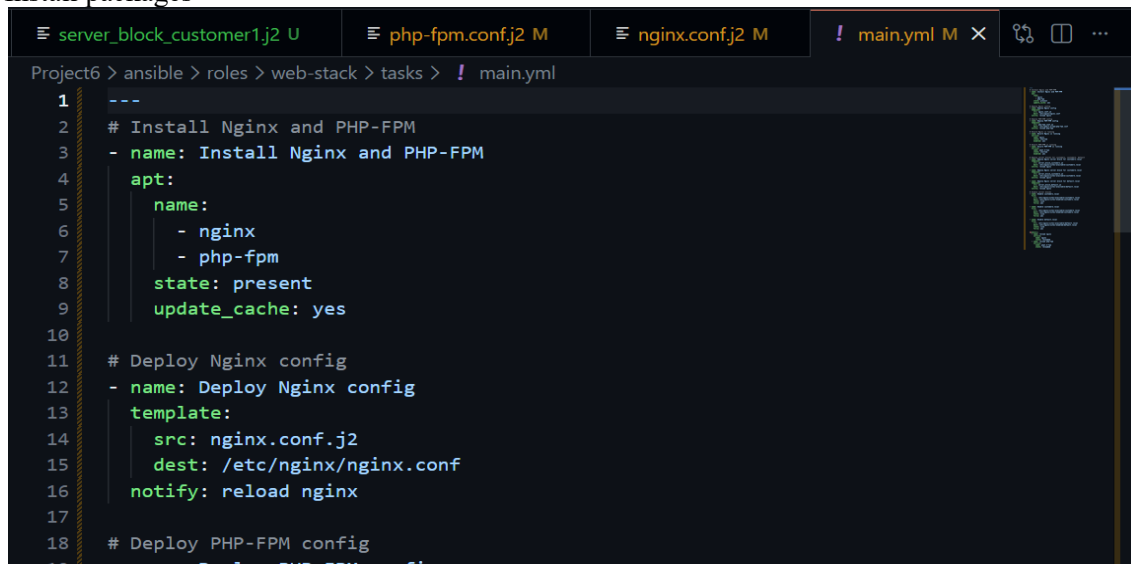
8.2 Web Stack Configuration (Nginx + PHP-FPM)

Ansible role `web-stack` installs and configures:

- Nginx
- PHP-FPM
- Performance tuning
- Multiple server blocks

Steps:

1. Install packages



```
Project6 > ansible > roles > web-stack > tasks > main.yml
1 ---
2 # Install Nginx and PHP-FPM
3 - name: Install Nginx and PHP-FPM
4   apt:
5     name:
6       - nginx
7       - php-fpm
8     state: present
9     update_cache: yes
10
11 # Deploy Nginx config
12 - name: Deploy Nginx config
13   template:
14     src: nginx.conf.j2
15     dest: /etc/nginx/nginx.conf
16     notify: reload nginx
17
18 # Deploy PHP-FPM config
19 - name: Deploy PHP-FPM config
```

2. Apply Nginx and PHP-FPM tuning

```
@hajrasarwar11 →/workspaces/CC-Project/Project6/ansible (main) $ ansible-playbook -i inventory/
aws_ec2.yml playbooks/provision-lb-nginx.yml
TASK [lb-nginx : Ensure Nginx is running] *****
changed: [ec2-3-29-129-53.me-central-1.compute.amazonaws.com]
changed: [ec2-3-29-138-95.me-central-1.compute.amazonaws.com]

RUNNING HANDLER [lb-nginx : reload nginx] *****
changed: [ec2-3-29-129-53.me-central-1.compute.amazonaws.com]
changed: [ec2-3-29-138-95.me-central-1.compute.amazonaws.com]

PLAY RECAP *****
ec2-3-29-129-53.me-central-1.compute.amazonaws.com : ok=11  changed=4  unreachable=0  fa
iled=0  skipped=0  rescued=0  ignored=0
ec2-3-29-138-95.me-central-1.compute.amazonaws.com : ok=11  changed=4  unreachable=0  fa
iled=0  skipped=0  rescued=0  ignored=0
```

3. Configure customer server blocks

```
server_block_customer1j2 U X  php-fpm.confj2 M  nginx.confj2 M  ! main.yml M  ...
Project6 > ansible > roles > web-stack > templates > server_block_customer1j2
1  server {
2      listen 80;
3      server_name customer1.local;
4      root /var/www/customer1;
5      index index.php index.html;
6      location ~ \.php$ {
7          include snippets/fastcgi-php.conf;
8          fastcgi_pass unix:/run/php/php7.4-fpm.sock;
9      }
```

8.3 Load Balancer Configuration

The lb-nginx role configures:

```
lb-nginx.confj2 M X
Project6 > ansible > roles > lb-nginx > templates > lb-nginx.confj2
1  user www-data;
2  worker_processes auto;
3  pid /run/nginx.pid;
4
5  http {
6      upstream web_backend {
7          server 3.28.184.165:80;
8          server 40.172.114.101:80 backup;
9      }
```

• Reverse proxy

```
CC-PROJECT [CODESPACES: friendly guacamole]
EXPLORER  PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
CC-PROJECT [CODESPACES: friendly guacamole]
  Project6
  ansible
    inventory
      ! aws_ec2.yml U
      ! production_aws_e...
      ! staging_aws_ec2.y...
      ! test_aws_ec2.yml
    playbooks
    roles
      lb-nginx
        handlers
        tasks
        templates
          lb-nginx.c... M
        monitoring
        web-stack
        handlers
        tasks
        main.yml M
        templates
@hajrasarwar11 →/workspaces/CC-Project/Project6/ansible (main) $ ansible role_lb -i inventor
y/aws_ec2.yml -a "cat /etc/nginx/nginx.conf"
proxy_cache_path /tmp/nginx_cache levels=1:2 keys_zone=my_cache:10m max_size=100m;

server {
    listen 80;
    server_name lb.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name lb.local;

    ssl_certificate /etc/nginx/ssl/nginx.crt;
    ssl_certificate_key /etc/nginx/ssl/nginx.key;

    location / {
        proxy_pass http://web_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_cache my_cache;
        add_header X-Cache-Status $upstream_cache_status;
    }
}
```

Nginx config

- SSL (self-signed)

```
@hajrasarwar11 →/workspaces/CC-Project/Project6/ansible (main) $ ansible role_lb -i inventory/aws_ec2.yml -a "ls -l /etc/nginx/ssl/"
[WARNING]: Host 'ec2-3-29-129-53.me-central-1.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ec2-3-29-129-53.me-central-1.compute.amazonaws.com | CHANGED | rc=0 >>
total 8
-rw-r--r--. 1 root root 1111 Jan 27 14:05 nginx.crt
-rw-----. 1 root root 1704 Jan 27 14:05 nginx.key
[WARNING]: Host 'ec2-3-29-138-95.me-central-1.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ec2-3-29-138-95.me-central-1.compute.amazonaws.com | CHANGED | rc=0 >>
total 8
-rw-r--r--. 1 root root 1111 Jan 27 14:05 nginx.crt
-rw-----. 1 root root 1704 Jan 27 14:05 nginx.key
@hajrasarwar11 →/workspaces/CC-Project/Project6/ansible (main) $
```

- HTTP → HTTPS redirect
- Cache with X-Cache-Status header
- Backup server behavior

8.4 Website Deployment & Onboarding Automation

Playbooks automate website deployment and onboarding.

Steps:

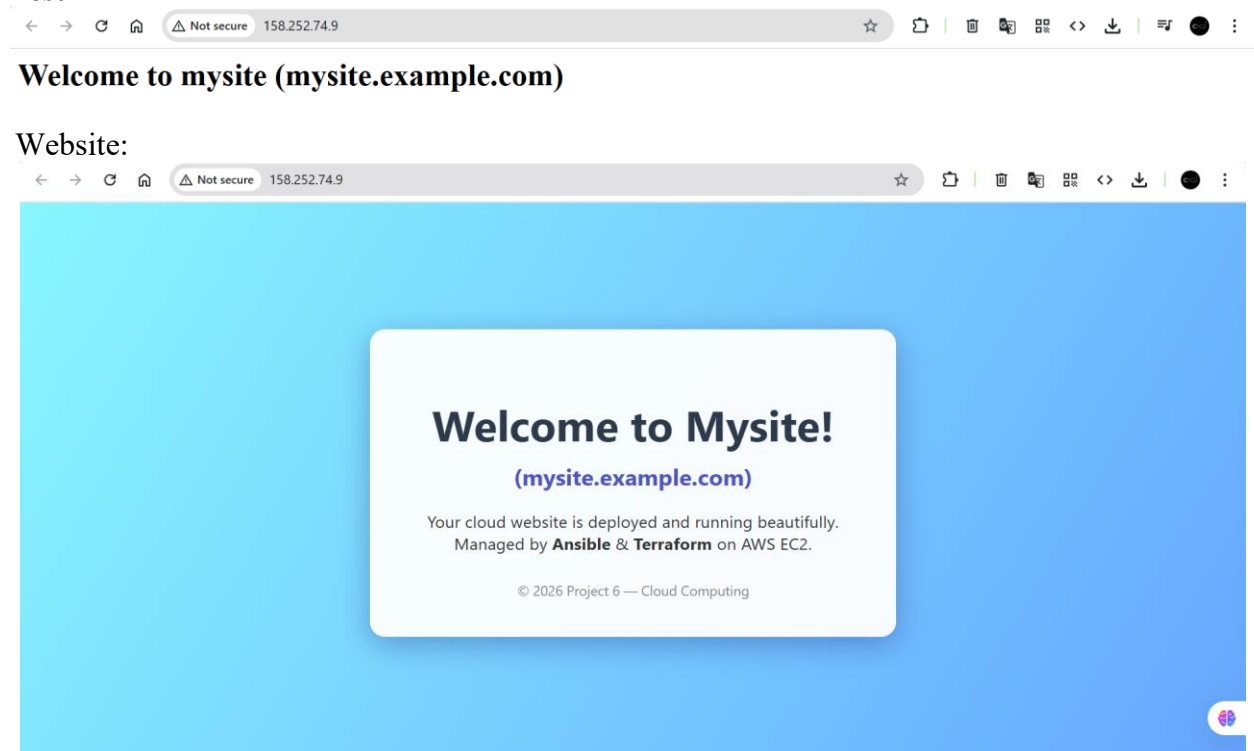
1. Deploy initial websites

```
@hajrasarwar11 →/workspaces/CC-Project/Project6/ansible (main) $ ansible-playbook -i inventory/aws_ec2.yml playbooks/add-website.yml --private-key=project6-key.pem -u ec2-user -e 'site_name=mysite server_name=mysite.example.com doc_root=/var/www/mysite enable_ssl=false'
[WARNING]: Host 'ec2-3-29-125-110.me-central-1.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [ec2-3-29-125-110.me-central-1.compute.amazonaws.com]

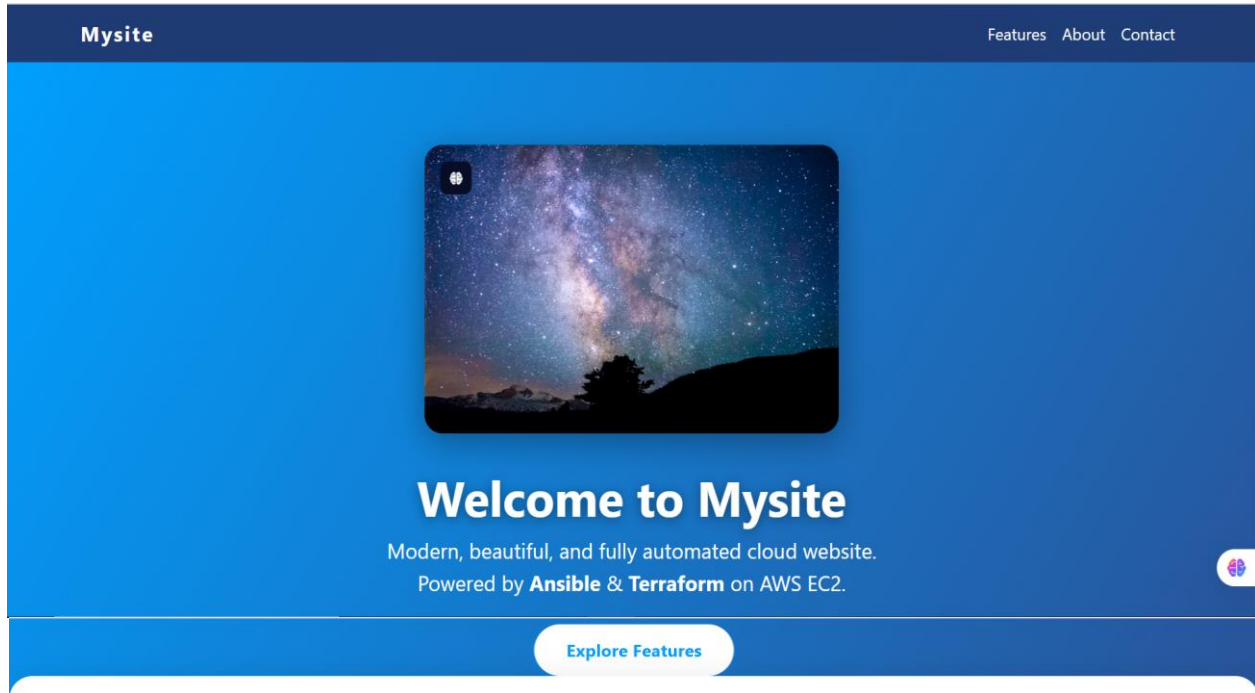
PLAY RECAP *****
ec2-158-252-74-9.me-central-1.compute.amazonaws.com : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ec2-3-29-125-110.me-central-1.compute.amazonaws.com : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

2. Add new customer site

Test:



Website2:



Our Features

Everything you need for a fantastic cloud presence.



Responsive Design

Looks great on any device, from desktop to mobile.



Lightning Fast

Deployed instantly with automation and cloud power.



Secure & Reliable

Hosted on AWS EC2 with best practices for security.



Automated by Ansible & Terraform

From infrastructure to deployment, everything is managed as code. No manual steps, just pure cloud magic!

[Learn More](#)

About This Project

This website was deployed automatically to the cloud using modern DevOps tools. It's scalable, secure, and easy to update. Perfect for demos, portfolios, or business landing pages.

[Contact Us](#)



Contact Us

Want a site like this? Reach out and let's build something amazing together!

[Email Us](#)

Final Deployed Website:

CloudLab

HomeFeaturesSecurityAboutCo

Cloud Innovation & Security Lab

Futuristic Cloud Solutions, AI Integration, and Security Awareness for the Next Generation

Explore InnovationLearn Security

CloudLab

HomeFeaturesSecurityAboutCo

Features



Live Cloud Status

Monitor real-time status of major cloud providers and services, ensuring uptime and reliability for your applications.



Trending Cloud Tech

Stay updated with the latest trends in cloud computing, AI, and automation with curated news and insights.



Interactive Demos

Experience hands-on cloud labs and AI-powered automation demos to boost your skills and knowledge.

CloudLab

HomeFeaturesSecurityAboutCo

Cloud Security Awareness



Security Tips

Learn best practices to secure your cloud infrastructure and protect your data from threats.



Latest News

Get the latest updates on cloud security incidents, vulnerabilities, and industry news.



Interactive Quizzes

Test your knowledge with fun and challenging quizzes on cloud security and best practices.

CloudLab

HomeFeaturesSecurityAboutCo

About

Our Mission

To empower individuals and organizations with cutting-edge cloud solutions and security knowledge, fostering innovation and resilience in the digital era.

Meet The Team



Amina Noor

Cloud Solutions Architect



Hajra Sarwar

AI & ML Specialist



Hamail Fatima

UI/UX Designer

158.252.74.9/?#about

Contact & Socials

Email: hlofaam1@gmail.com



© 2026 Cloud Innovation & Security Lab. All rights reserved.

9. Monitoring & Customer Onboarding

9.1 Monitoring Scripts

A Bash script checks:

- HTTP status code

```
$ check-site.sh M x
Project6 > ansible > roles > monitoring > files > $ check-site.sh
1  #!/bin/bash
2  # Script to check HTTP status and response time for each site
3  # Logs to /var/log/site_checks.log
4
5  SITES=(
6      "http://localhost"
7      "http://127.0.0.1"
8      # Add more sites as needed, e.g. "http://customer1.example.com"
9  )
10
11 LOGFILE="/var/log/site_checks.log"
12 DATE=$(date '+%Y-%m-%d %H:%M:%S')
13
14 for SITE in "${SITES[@]"; do
15     RESULT=$(curl -o /dev/null -s -w "%{http_code} %{time_total}" "$SITE")
16     STATUS=$(echo $RESULT | awk '{print $1}')
```

- Response time
- Logs results via cron

```
@hajrasarwar11 → /workspaces/CC-Project/Project6/ansible (main) $ sudo bash roles/monitoring/files/
check-site.sh
@hajrasarwar11 → /workspaces/CC-Project/Project6/ansible (main) $ cat /var/log/site_checks.log
2026-01-27 19:16:52 | http://localhost | status: 000 | response_time: 0.000175s
2026-01-27 19:16:52 | http://127.0.0.1 | status: 000 | response_time: 0.000085s
```

9.2 Customer Onboarding Documentation

Documentation explains:

- Required inputs
- Naming conventions
- SSL handling
- Verification steps

```
CC-PROJECT [CODESPACES: friendly guacamole]
EXPLORER
  CC-PROJECT [CODESPACES: friendly guacamole]
  Project6
  ansible
  aws
  docs
    architecture.md
    customer-onb... IM
    monitoring.md
    performance-tunin...
    ssl-strategy.md
  terraform
  websites
  .gitignore IM
  awscli2.zip U
  README.md IM
  README.md
customer-onboarding.md M x
Project6 > docs > customer-onboarding.md > abc # Customer Onboarding Guide > abc ## 3. How to Verify via Load Balancer (t
1  # Customer Onboarding Guide
2
3  This guide explains how to onboard a new customer website to the Project 6
4  cloud platform.
5
6  ## 1. Inputs Needed to Add a Site
7  To add a new site, you need the following information:
8  - **site_name**: A unique identifier for the site (e.g., `customer1`)
9  - **server_name**: The domain name for the site (e.g., `customer1.example.com`)
10 - **doc_root**: The document root directory for the site (e.g., `/var/www/
11   customer1`)
12 - **enable_ssl**: Whether to enable SSL (true/false)
13
14 ## 2. Running the Playbook
15 To add a new website, run the following command from the `ansible` directory:
16
17 ```bash
18 ansible-playbook -i inventory/aws_ec2.yml playbooks/add-website.yml \
19 -e "site_name=customer1 server_name=customer1.example.com doc_root=/var/www/
20   customer1 enable_ssl=false"
```

10 Conclusion

This project successfully demonstrates a **high-performance, multi-tenant web hosting platform** using **AWS, Terraform, Ansible, and Nginx**.

The solution is **scalable, automated, secure, and aligned with real-world DevOps practices**, while strictly adhering to the course scope.
